DOCKER-CONTAINER-MANAGEMENT

Prepared by
Avishek Paudel


avishekspaudel12@gmail.com

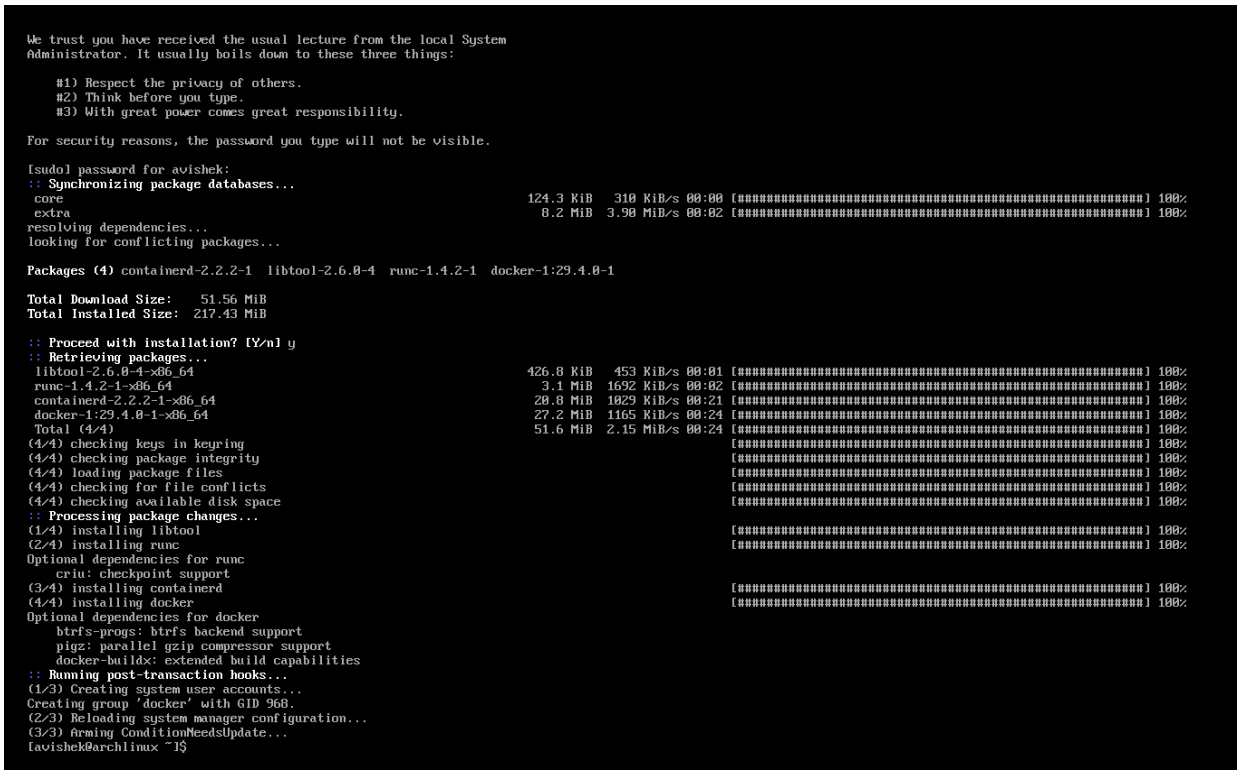
Table of Contents

Introduction

This report documents the steps performed during Docker workshop 5. The objective of this workshop was to install Docker, run containers, manage them, and verify functionality through practical commands. Screenshots are included to provide evidence of successful execution.

Install Docker

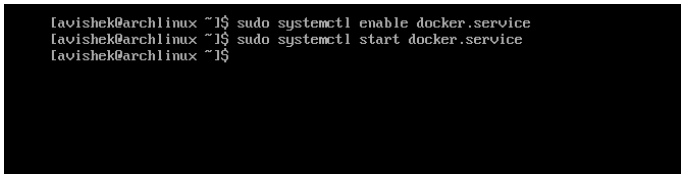
Command: sudo pacman -Syy docker
This installs Docker using Arch Linux's package manager



Installed.

Start Docker Service

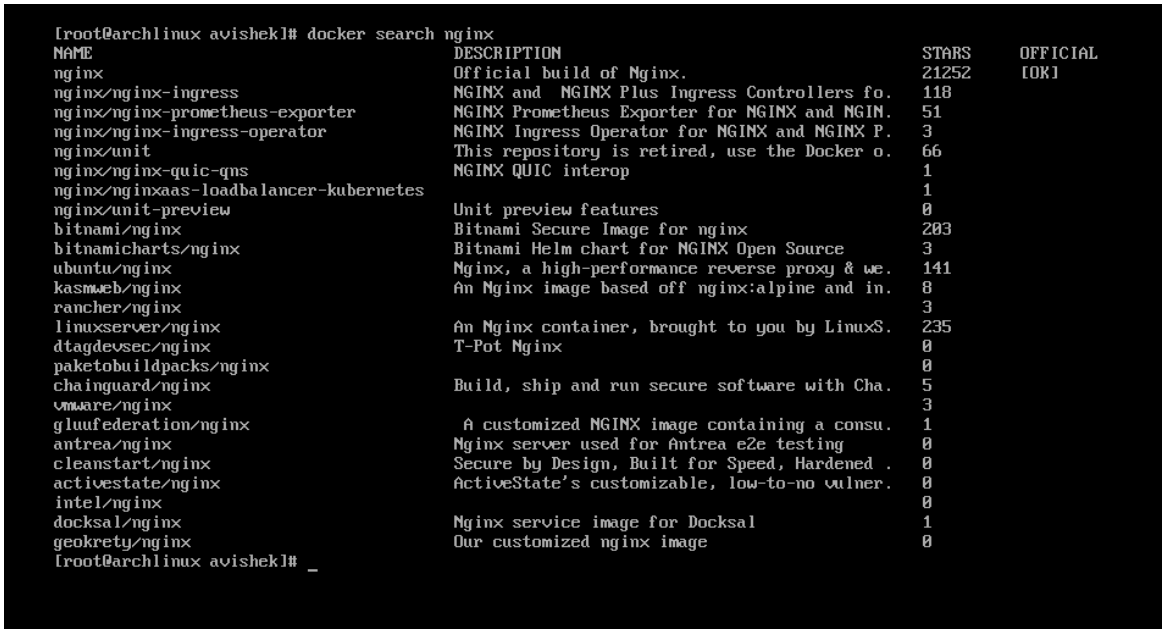
Command:
sudo systemctl enable docker.service = enables Docker at boot.
sudo systemctl start docker.service = starts Docker immediately.
Docker runs as a background service called the Docker daemon.



Search Image

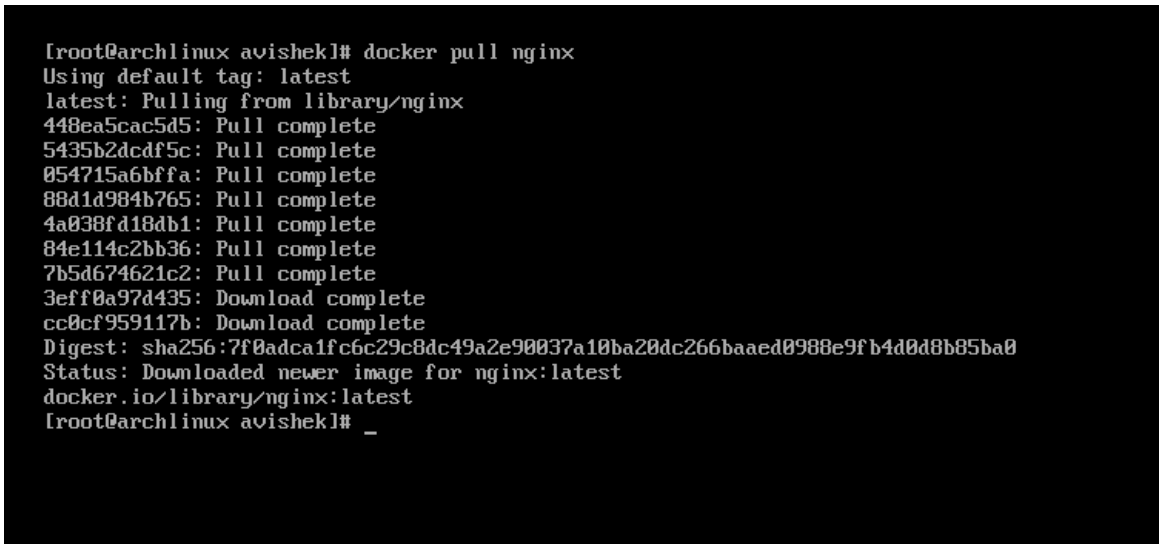
Command: docker search nginx

- Searches Docker Hub for available nginx images.



Pull Image

Command: docker pull nginx
- Downloads the nginx image locally.



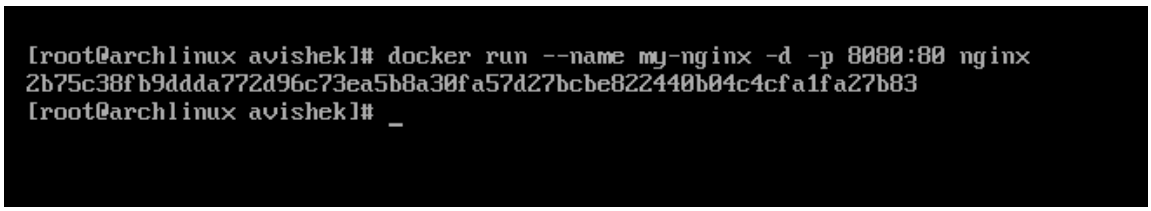
Run Container

Command: docker run –name my-nginx -d -p 8080:80 nginx

- Creates and runs a container named my-nginx.

- -d runs in detached mode

- -p 8080:80 maps host port 8080 to container port 80.



Check Running Containers

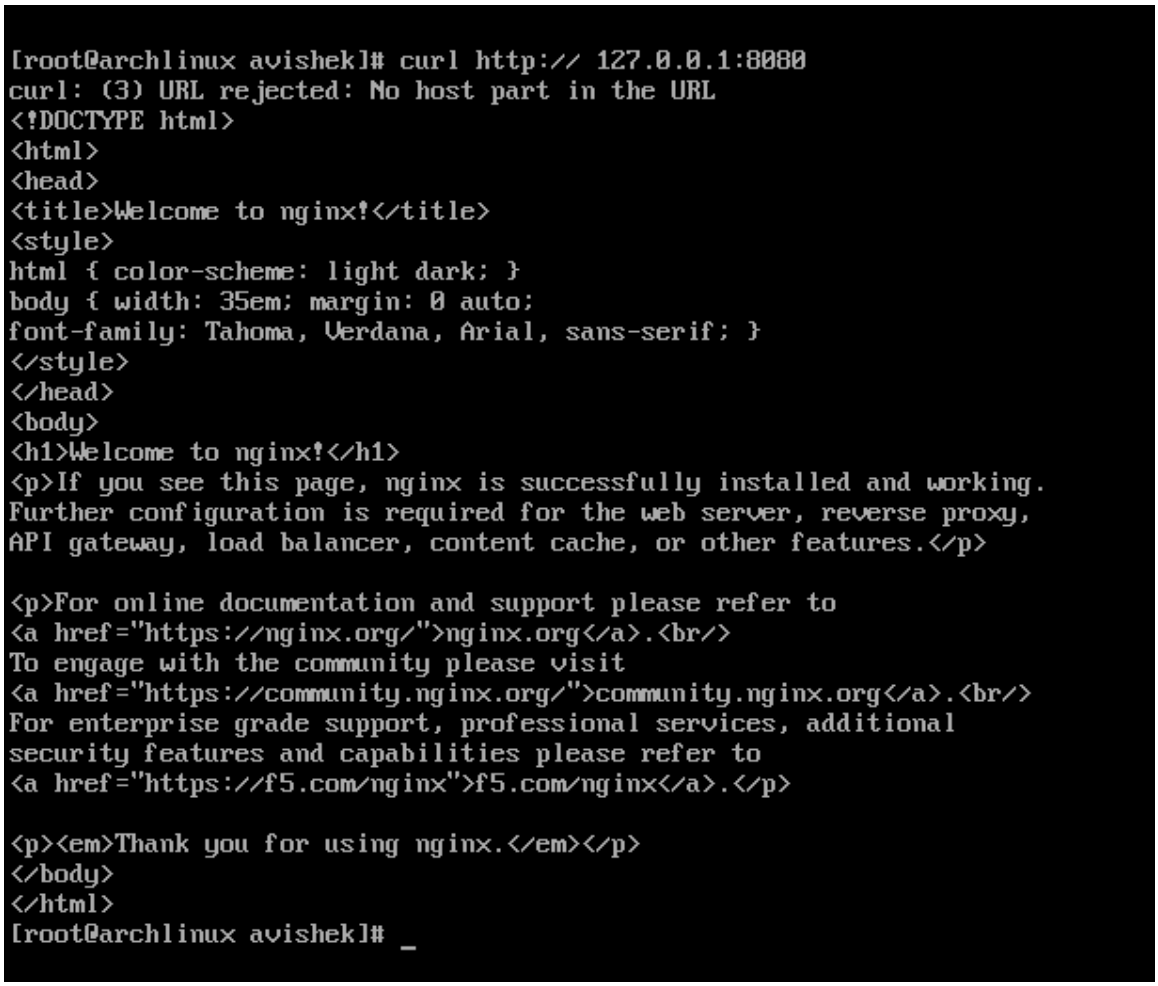
Command: docker ps

- lists active containers with details like ID, name, ports and status

Test Container

Command: curl http://127.0.0.1:8080

- Confirms nginx is serving content via port mapping.



Execute Command Inside Container

Command: docker exec my-nginx cat
/usr/share/nginx/html/index.html
- Display the default nginx HTML file inside the container.

```
[root@archlinux avishek]# docker exec my-nginx cat /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
[root@archlinux avishek]#
```

Run Second Container

Command: docker run –name my-nginx2 -d -p 8081:80 nginx
- Demonstrate running multiple containers simultaneously.

```
[root@archlinux avishek]# docker run --name my-nginx2 -d -p 8081:80 nginx
6bdce26369a172b78a73035af2a21da8510212f44db40d63161683f2d38507f8
[root@archlinux avishek]#
```

Stop / Start / Remove

Commands:

docker stop my-nginx = stops the containers

docker start my-nginx = restarts it

docker rm my-nginx2 = removes the second container

```
[root@archlinux avishek]# docker stop my-nginx
my-nginx
[root@archlinux avishek]# docker start my-nginx
my-nginx
[root@archlinux avishek]# docker stop my-nginx2
my-nginx2
[root@archlinux avishek]# docker rm my-nginx2
my-nginx2
[root@archlinux avishek]# _
```

Hello World Test

Commands:

1. docker pull hello-world
2. docker run hello-world

```
[root@archlinux avishek]# docker pull hello-world
Using default tag: latest
dockerlatest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest
[root@archlinux avishek]# docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

[root@archlinux avishek]# _
```